Docker 內部網路

大綱

1. 基本概念
2. 網路驅動程式
3. bridge 網路
4. host 網路
5. overlay 網路
6. none 網路
7. 自訂網路
8. 容器間通訊
9. 連接埠映射
10. DNS 與服務發現
11. 網路命令
12. 實際範例

---

1. 基本概念

1.1 Docker 網路架構

Docker 容器網路是 Docker 解決方案的核心功能，允許容器之間以及容器與主機之間進行通訊。

```
┌─────────────────────────────────────────────────┐
│              Docker Host                        │
│                                                 │
│  ┌──────────────────────────────────────┐       │
│  │         Docker Engine                │       │
│  │  ┌──────────┐  ┌──────────────┐      │       │
│  │  │  bridge  │  │    host      │      │       │
│  │  │  network │  │    network   │      │       │
│  │  └──────────┘  └──────────────┘      │       │
│  │        │                             │       │
│  │        │                             │       │
│  │  ┌─────▼────┐                        │       │
│  │  │ container │◄── eth0               │       │
│  │  │  network  │                       │       │
│  │  └──────────┘                        │       │
│  └──────────────────────────────────────┘       │
│                                                 │
│  └── docker0: 172.17.0.1                        │
└─────────────────────────────────────────────────┘
```

1.2 網路驅動程式類型

驅動程式	說明	使用場景
`bridge`	預設橋接網路	單一主機容器通訊
`host`	移除網路隔離	效能敏感場景
`overlay`	跨主機容器通訊	Swarm 叢集
`none`	無網路	隔離容器
`macvlan`	指定 MAC 位址	傳統應用程式
`ipvlan`	IPv4/IPv6 子網路	特殊網路需求

1.3 預設網路

```
# 查看預設網路
docker network ls

# 輸出範例:
# NETWORK ID     NAME              DRIVER    SCOPE
# xxxxx          bridge            bridge    local
# xxxxx          host              host      local
# xxxxx          none              null      local
```

---

2. 網路驅動程式

2.1 驅動程式特性

驅動程式	網路隔離	跨主機	NAT	DHCP
bridge	是	否	是	Docker
host	否	否	無	主機
overlay	是	是	是	Docker
none	完全隔離	否	無	無

2.2 選擇指南

- **單一主機**：使用 `bridge` 或自訂 bridge
- **效能敏感**：使用 `host`
- **多主機叢集**：使用 `overlay`
- **需要固定 IP**：使用 `macvlan` 或 `ipvlan`

---

3. bridge 網路

3.1 預設 bridge

Docker 安裝後會自動建立 `bridge` 網路，連接到此網路的容器使用 172.17.0.0/16 子網路。

```
# 查看 bridge 網路詳細資訊
docker network inspect bridge

# 預設閘道通常是 172.17.0.1
```

3.2 容器連接到預設 bridge

```
# 啟動容器（自動連接至預設 bridge）
docker run -d nginx

# 手動連接
docker run -d --network bridge nginx

# 查看容器 IP
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
<container>
```

3.3 預設 bridge 限制

- 無法使用 DNS 自動解析容器名稱
- 所有容器必須手動連結（--link）或有共通的網路
- 無法對容器進行網路隔離分組

3.4 為什麼要避免使用預設 bridge

```
# 問題：無法透過容器名稱 DNS 解析
docker run -d --name web nginx
docker run -d --name app myapp

# 在 app 容器內無法 ping web，需要使用 --link（已淘汰）
docker run -d --name app --link web myapp

# 解決方案：使用自訂 bridge 網路
docker network create my-net
docker run -d --name web --network my-net nginx
docker run -d --name app --network my-net myapp

# 現在 app 可以解析 web
```

---

4. host 網路

4.1 特性

容器直接使用主機網路命名空間，移除網路隔離層。

4.2 使用方式

```
# 啟動容器使用主機網路
docker run -d --network host nginx

# 注意：不需要 -p 映射，因為容器直接使用主機連接埠
```

4.3 優缺點

優點	缺點
最高效能	無網路隔離
無 NAT 延遲	連接埠衝突風險
簡單設定	難以管理

4.4 適用場景

- 效能關鍵應用
- 需要全權控制網路
- 網路監控工具

---

5. overlay 網路

5.1 概念

overlay 網路實現跨 Docker 主機的容器通訊，用於 Swarm 叢集。

5.2 前提條件

- Docker Engine 19.03+
- 叢集模式（Swarm）或相容的密鑰值儲存
- 沒有防火牆阻止叢集通訊

5.3 建立 overlay 網路

```
# Swarm 模式下建立
docker network create -d overlay my-overlay

# 建立加密的 overlay 網路
docker network create -d overlay --encrypt my-overlay-encrypted
```

5.4 工作原理

```
Host A                          Host B
┌──────────────┐                ┌──────────────┐
│  Container   │◄── VXLAN ──────►│  Container   │
│  (172.18.0.2)│                 │  (172.18.0.3)│
└──────────────┘                └──────────────┘
      │                               │
┌─────┴────────┐                ┌────┴─────────┐
│  docker_gwbridge │            │  docker_gwbridge │
└──────────────┘                └──────────────┘
```

---

6. none 網路

6.1 特性

容器完全隔離，無網路介面（除了 loopback）。

6.2 使用方式

```
# 建立無網路容器
docker run -d --network none nginx

# 進入容器確認
docker exec -it <container> ip addr

# 輸出只會有 lo (127.0.0.1)
```

6.3 適用場景

- 安全隔離
- 只需要本地存取
- 批次處理

---

7. 自訂網路

7.1 建立 bridge 網路

```
# 建立自訂 bridge 網路
docker network create --driver bridge my-bridge

# 指定子網路
docker network create --driver bridge \
    --subnet 192.168.10.0/24 \
    my-bridge

# 指定閘道
```

```
docker network create --driver bridge \
    --subnet 192.168.10.0/24 \
    --gateway 192.168.10.1 \
    my-bridge
```

7.2 建立完整範例

```
# 建立網路
docker network create \
    --driver bridge \
    --subnet 192.168.100.0/24 \
    --gateway 192.168.100.1 \
    app-net

# 建立容器並連接
docker run -d --name web --network app-net nginx
docker run -d --name db --network app-net mysql:8.0

# 驗證網路功能
docker exec web ping db
```

7.3 網路參數

參數	說明
`--driver`	網路驅動程式（預設 bridge）
`--subnet`	子網路 CIDR 格式
`--gateway`	閘道 IP
`--ip-range`	IP 位址範圍
`--internal`	限制內部通訊
`--attachable`	允許獨立容器連接（overlay）

7.4 內部網路

```
# 建立只能內部使用的網路
docker network create --driver bridge --internal app-internal
```

---

8. 容器間通訊

8.1 同一網路內通訊

同一自訂網路中的容器可以透過容器名稱 DNS 解析：

```
# 建立網路
docker network create app-net

# 啟動兩個容器
docker run -d --name web --network app-net nginx
docker run -d --name app --network app-net myapp

# 在 app 容器內測試
docker exec app ping web
# 輸出: PING web (172.18.0.2) 56(84) bytes of data.
```

8.2 跨網路通訊

不同網路的容器需要透過路由器或外部網路連接:

```
# 建立兩個網路
docker network create net1
docker network create net2

# 連接容器到不同網路
docker run -d --name web --network net1 nginx
docker run -d --name db --network net2 mysql:8.0

# 連接現有容器到新網路
docker network connect net2 web
```

8.3 斷開網路連接

```
# 斷開容器與網路的連接
docker network disconnect net1 web
```

---

9. 連接埠映射

9.1 基本映射

```
# 格式: 主機連接埠:容器連接埠
docker run -d -p 80:80 nginx

# 多個連接埠
docker run -d -p 80:80 -p 443:443 nginx

# 指定主機 IP
docker run -d -p 192.168.1.100:80:80 nginx
```

9.2 映射選項

選項	說明
`-p 8080:80`	對應 TCP 80
`-p 8080:80/udp`	對應 UDP 80
`-p 192.168.1.100:80:80`	對應特定 IP
`-P`	發布所有 EXPOSE 的連接埠

9.3 發布模式

```
# 輸入模式（ingress）- Swarm 負載平衡
docker run -d -p mode=ingress,target=80,published=80 nginx

# 主機模式（host）- 直接連接
docker run -d -p mode=host,target=80,published=80 nginx
```

9.4 查看映射

```
# 查看容器連接埠
docker port <container>

# 輸出範例:
# 80/tcp -> 0.0.0.0:80
# 443/tcp -> 0.0.0.0:443
```

---

10. DNS 與服務發現

10.1 內建 DNS

Docker 提供內建 DNS 伺服器（127.0.0.11），用於解析容器名稱。

10.2 DNS 解析規則

名稱類型	解析方式
容器名稱	自動 DNS 解析
自訂網路	可互相解析
預設 bridge	需要 --link
overlay 網路	自動解析

10.3 DNS 選項

```
# 使用自訂 DNS 伺服器
docker run -d --dns 8.8.8.8 nginx

# 新增 DNS 搜尋尾碼
docker run -d --dns-search example.com nginx

# 新增主機記錄
docker run -d --add-host myhost:192.168.1.1 nginx
```

10.4 驗證 DNS

```
# 進入容器測試 DNS
docker exec -it <container> nslookup web
docker exec -it <container> cat /etc/resolv.conf
```

---

11. 網路命令

11.1 基本命令

```
# 列出所有網路
docker network ls

# 查看網路詳細資訊
docker network inspect bridge
docker network inspect my-net

# 建立網路
docker network create my-net
docker network create --driver bridge my-bridge

# 移除網路
docker network rm my-net

# 清除未使用的網路
docker network prune
```

11.2 網路連接命令

```
# 連接容器到網路
docker network connect my-net web

# 斷開容器與網路的連接
docker network disconnect my-net web
```

```
# 查看容器連接的網路
docker inspect -f '{{range .NetworkSettings.Networks}}{{.NetworkID}}{{end}}' web
```

11.3 進階網路操作

```
# 建立帶有特定子網路的網路
docker network create \
    --driver bridge \
    --subnet 172.28.0.0/16 \
    --gateway 172.28.0.1 \
    --ip-range 172.28.5.0/24 \
    custom-net

# 建立隔離的內部網路
docker network create --driver bridge --internal isolated-net

# 建立 overlay 網路
docker network create --driver overlay --attachable my-overlay
```

---

12. 實際範例

12.1 基本 Web 應用程式網路架構

```
# 1. 建立網路
docker network create app-net

# 2. 啟動 Nginx（前端）
docker run -d \
    --name nginx \
    --network app-net \
    -p 80:80 \
    nginx:alpine

# 3. 啟動 Node.js（後端 API）
docker run -d \
    --name api \
    --network app-net \
    -e DB_HOST=db \
    my-api:latest

# 4. 啟動 MySQL（資料庫）
docker run -d \
    --name db \
    --network app-net \
    -e MYSQL_ROOT_PASSWORD=secret \
    mysql:8.0

# 5. 驗證網路連接
```

```
docker exec nginx ping api
docker exec api ping db
```

12.2 多環境網路隔離

```
# 建立開發和生產網路
docker network create --driver bridge dev-net
docker network create --driver bridge prod-net

# 部署到開發網路
docker run -d --name web-dev --network dev-net nginx
docker run -d --name api-dev --network dev-net myapi:dev

# 部署到生產網路
docker run -d --name web-prod --network prod-net nginx
docker run -d --name api-prod --network prod-net myapi:prod
```

12.3 Swarm 多主機網路

在 Swarm 模式下使用 overlay 網路：

```
# 初始化 Swarm
docker swarm init

# 建立 overlay 網路（可附加）
docker network create -d overlay --attachable app-overlay

# 部署服務
docker service create --name web \
    --network app-overlay \
    --replicas 3 \
    -p 80:80 \
    nginx:latest
```

12.4 網路除錯

```
# 檢查容器網路設定
docker inspect -f '
    NetworkSettings.Networks: {{range $key, $value := .NetworkSettings.Networks}}
        {{$key}}: IP={{.IPAddress}} Gateway={{.Gateway}}
    {{end}}
' <container>

# 測試網路連線
docker exec -it <container> ping <target>

# 追蹤網路路徑
```

```
docker exec -it <container> traceroute <target>

# 查看 DNS 解析
docker exec -it <container> nslookup <hostname>
```

總結

- **預設 bridge**：功能有限，適合簡單場景
- **自訂 bridge**：支援 DNS，適合單一主機應用
- **host 網路**：最高效能，無隔離
- **overlay 網路**：跨主機通訊，Swarm 必備
- **none 網路**：完全隔離
- **連接埠映射**：容器對外暴露服務
- **DNS 解析**：容器名稱自動解析